PROJECT 3 • TECHNICAL GUIDE

Technical Specifications
Quest Study Application

A standalone technical guide for building the application — intended for an agent in an integrated development environment (IDE). It contains only the specifications, AI rules, API design, and database design needed for the build.

This document is the authoritative reference. Do not replace the technology stack, versions, or constraints in it with alternatives without the team's approval.

1. Application and locked scope

Quest Study is a gamified study web application for senior high school students. Students complete quests to earn experience points (XP), maintain study streaks, and collect badges. Timed quizzes are presented as boss fights.

- **Two quest types:** Assignment Quests (set by the teacher, deadline-driven) and Revision Quests (set by the student, exam-preparation-driven), running on the same engine.
- **Teacher-lite:** teachers create quests and quizzes and view students' progress and quiz scores. There is no file submission by students and no file grading by teachers in the application.
- **Full CRUD:** create, read, update, and delete operations for quests, quizzes, and related data, entirely through the web service.
- **Gamification:** XP, a level-up every 200 XP, streaks, and badges.
- **Architecture:** React (teacher / student) → Django REST Framework API → Django → PostgreSQL, and for AI: Django → local Ollama. The frontend never accesses PostgreSQL directly.

2. Software specifications

Component	Specification	Notes
Operating system	Windows 11	Development machine and host during development; a laptop serves as host until finalization
Programming language	Python 3.11 or later	Primary language for the Django backend
Backend	Django and Django REST Framework	REST API web service in JSON format; all frontend data access passes through this layer
Frontend	React	Responsive web application for phone and desktop browsers; no installation required for users
Database	PostgreSQL 16 or later	Primary database; Django JSONField is stored as JSONB for AI-generated quiz structure
Artificial intelligence	Ollama — Qwen2.5 3B Instruct	Runs locally on the server side; inference only
Development tools	Git, Python virtual environment, VS Code or equivalent	Git for version control; Python dependencies are isolated in a virtual environment
Client	Modern browser — Chrome, Edge, or Firefox	No installation; no local AI model runs on the student's device

AI rules that must be followed

The following constraints are part of the specification decision and must not be violated in the build:

- **Local inference only:** Ollama is used only for inference running locally. There is no training and no fine-tuning of the model.
- **No subscription APIs:** do not use any subscription or paid AI API in any form. All generation is done locally.
- **Generated once, shared by all:** the teacher generates a quiz once for each topic, and students use the stored cached result, rather than regenerating it for each student.
- **Validation and fallback:** AI output must be validated as JSON. If it is invalid, make one retry; if it still fails, use a template quiz bank as the fallback.
- **Server-side only:** Ollama is called only by Django from the server side. The browser and frontend never call Ollama directly.

3. Hardware specifications

Development machine and host (laptop)

During development, the laptop serves as both the development machine and the application host, including running PostgreSQL and Ollama locally.

Level	CPU	RAM	Storage	GPU
Minimum	Multi-core	8 GB	20 GB free SSD	Not required
Recommended	Multi-core	16 GB	50 GB free SSD	Optional: GPU with 6–8 GB VRAM, only for faster AI generation

A graphics processing unit (GPU) is not required to run the Qwen2.5 3B Instruct model, because the model can run on CPU. A GPU with 6–8 GB of VRAM is optional and only speeds up quiz generation; it is not a requirement for the application to run.

User devices — students and teachers

Students and teachers do not need a special device. Any laptop or phone with a modern browser and internet access is sufficient, with about 2 GB of RAM available for the browser. No application needs to be installed and no AI model runs on the user's device, because all AI processing happens on the server side.

4. Web service design (REST API)

All endpoints are under /api/, use HTTP and JSON format, and are built with Django REST Framework. The following list is a planned design, not code that has already been built. There are no endpoints for file submission or file grading, in line with the teacher-lite model. The AI endpoints call Ollama from the server side; the browser never calls Ollama directly.

Authentication

Method and endpoint	Description
POST /api/auth/register/	Account registration
POST /api/auth/login/	Sign in (login)
GET /api/auth/me/	Profile of the currently signed-in user

Quests

Method and endpoint	Description
GET /api/quests/	List quests; filterable by type=assignment|revision, subject, status
POST /api/quests/	Create a quest (teacher)
GET /api/quests/{id}/	Details of one quest
PATCH /api/quests/{id}/	Update a quest (teacher)
DELETE /api/quests/{id}/	Delete a quest (teacher)

Quizzes (boss fights)

Method and endpoint	Description
GET /api/quizzes/	List quizzes
POST /api/quizzes/	Create a quiz (teacher)
GET /api/quizzes/{id}/	Details of one quiz
POST /api/quizzes/{id}/attempts/	Submit a quiz attempt (student)
GET /api/quizzes/{id}/attempts/{attempt_id}/	View one attempt and its score

Progress and XP

Method and endpoint	Description
GET /api/progress/me/	The student's own progress: XP, streaks, and badges
GET /api/progress/	Teacher view of students' progress and quiz scores

AI — server-side only

Method and endpoint	Description
POST /api/ai/generate-quiz/	Django asks local Ollama to generate a quiz for a topic, then stores the result in cache
POST /api/ai/revision-plan/	Django asks local Ollama to prepare a revision plan

5. Database design

The structure of AI-generated quizzes varies, so it is stored with Django's JSONField, which is stored as JSONB in PostgreSQL.

Table	Main contents
users	Teacher and student accounts, including roles
quests	Assignment Quests and Revision Quests, deadlines, status, progress
quizzes and questions	Quizzes and their questions, including AI-generated structure in JSONB
quiz_attempts	Student quiz attempts, answers, scores, time

xp_events	A record of every XP award through one central function
streaks	Study streak record for each student
badges	Badges earned by students

6. Deferral notes

The final hosting and deployment specifications are intentionally deferred and will only be decided at the finalization stage, in line with the decision already made. The laptop remains the host until that stage.

The PostgreSQL setup on Windows 11 remains parked. That setup does not begin until the user and the supervisor agree on the proposal and specifications.